Decision Transformer Warm-Starts for Minimum-Time Vehicle Trajectory Optimization with Obstacle Avoidance

Victor Sebastian Martinez Perez [1]

Abstract

Reliable and efficient trajectory optimization is critical for autonomous racing, where nonlinear dynamics, obstacle avoidance, and tight runtime constraints make online planning difficult. Learning-based methods offer a promising way to augment traditional optimization pipelines, but they often either replace the optimizer entirely or are judged only by imitation accuracy. This project studies whether a Decision Transformer can provide useful warm-starts for nonlinear minimum-time vehicle trajectory optimization with obstacle avoidance. Although larger-context training and targeted fine-tuning improve offline validation performance, these gains do not yield a convincing downstream benefit: warm-start acceptance remains near zero under the default gate, iteration reduction is negligible, and no model clearly improves total runtime once inference cost is included. Overall, the results show that offline sequence-model quality alone is not a reliable indicator of warm-start utility in this setting.

1. Introduction and Related Work

Trajectory optimization is a core component of autonomous racing and high-performance vehicle planning. Near the handling limits, the planner must account for nonlinear vehicle dynamics, track boundaries, actuator limits, and obstacle avoidance while still operating under tight runtime constraints. This makes optimization-based planning attractive for its ability to encode dynamics and constraints directly, but also challenging in practice because performance depends strongly on formulation, initialization, and numerical conditioning. These issues are well known in autonomous racing and nonlinear model predictive control, where real-time solution quality can depend strongly on both model fidelity and the quality of the initial guess (Kapania et al., 2015; Subosits & Gerdes, 2020; 2021; Waechter & Biegler, 2006).

These challenges motivate learning-based methods that can reduce online computational burden or provide better initial guesses to a downstream solver. However, existing learning-based approaches often fall into one of two categories. Some attempt to replace optimization altogether, which can weaken hard constraint satisfaction and robustness in safety-critical settings. Others imitate optimizer outputs through supervised learning, but are evaluated mainly through prediction error rather than through their effect on downstream optimization. In this project, the learned model is a Decision Transformer (DT), which is naturally viewed as an offline reinforcement learning method: it is trained on previously collected trajectories to model action generation conditioned on past trajectory context and trajectory-level objectives, and it is used as a policy at deployment time (Chen et al., 2021). For warm-starting applications, the relevant question is therefore not whether the model predicts expert trajectories accurately under teacher forcing, but whether it improves the operating point of the downstream optimizer.

This project studies that question in the context of nonlinear minimum-time vehicle trajectory optimization with obstacle avoidance. The learned component is a DT trained offline on trajectories generated by a direct-collocation solver. At deployment time, the DT produces an autoregressive warm-start trajectory, which is then validated, optionally projected, and passed to the downstream nonlinear program. This setup follows the broader idea of learned optimizer priors: rather than replacing optimization, the learned model is used to improve initialization while the solver remains responsible for hard constraints and final refinement (Guffanti et al., 2024; Celestini et al., 2024).

The work is most closely related to three lines of prior literature. First, autonomous racing and aggressive driving have long relied on trajectory optimization and model predictive control formulations that balance model fidelity, runtime, and robustness near the limits of handling (Kapania et al., 2015; Subosits & Gerdes, 2020; 2021). Second,

[1]Department of Aeronautics and Astronautics, Stanford University, Stanford, CA, USA. Correspondence to: Victor Sebastian Martinez Perez <sebasmp@stanford.edu>.

Proceedings of the 35^{th} International Conference on Machine Learning, Stockholm, Sweden, PMLR 80, 2018.

Decision Transformer and related sequence-modeling approaches show that offline control can be cast as conditional sequence prediction, often producing strong behavioral policies from fixed datasets (Chen et al., 2021). Third, recent transformer-based warm-starting methods for optimal control suggest that generative models can provide useful initializations to downstream solvers while preserving solver-enforced feasibility (Guffanti et al., 2024; Celestini et al., 2024). These results make learned warm-starting a plausible strategy, but they do not imply that improved offline imitation will necessarily yield reduced solver effort in nonlinear vehicle planning.

The specific goal of this project is to evaluate whether improved offline trajectory modeling yields improved warm-start utility in a constrained racing setting. The paper makes three contributions. First, it implements an end-to-end DT warm-start pipeline for nonlinear minimum-time vehicle trajectory optimization with obstacle avoidance, combining offline sequence modeling with rollout validation, projection, and solver refinement. Second, it develops targeted repair-data interventions, including post-projection and failure-conditioned fine-tuning, to address rollout-induced errors that do not appear in nominal expert data. Third, it shows through corrected deployment evaluations that improved offline sequence-model quality does not automatically yield improved warm-start utility in this setting, and identifies early lateral rollout behavior as the dominant remaining bottleneck. The remainder of the paper describes the approach in Section 2, the experimental setup in Section 3, the empirical findings in Section 4, the failure analysis in Section 5, and final limitations and conclusions in Sections 6 and 7.

2. Approach

2.1. Optimal control problem

The downstream planner solves a nonlinear minimum-time trajectory optimization problem for a racing vehicle on a closed track with optional static circular obstacles. The optimizer uses a nonlinear single-track vehicle model with load-transfer states and Fiala brush tire forces, parameterized to the Volkswagen Golf GTI test vehicle used in prior Stanford autonomous-racing studies (Subosits & Gerdes, 2020; 2021). The dynamic-model implementation reused and adapted prior Dynamics Design Lab vehicle-model code, and the optimization layer was implemented in Python with CasADi (Andersson et al., 2019) and the FATROP solver path used in the final project phase. The project assumes dry-asphalt operating conditions, consistent with the nominal tire-model setting used in this planning stack. The problem is posed in the spatial domain and discretized by direct collocation over a horizon of $N = 120$ nodes on the Oval track. The benchmark geometry is shown in Figure 1.

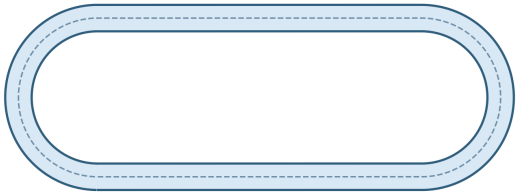

Figure 1. Oval-track benchmark geometry, with length of 260 m and track width of 6 m.

Let the state and control at node k be

$$x_k = [u_{x,k},\, u_{y,k},\, r_k,\, \Delta F_{z,\ell,k},\, \Delta F_{z,a,k},\, t_k,\, e_k,\, d\psi_k]^\top$$
$$u_k = [\delta_k,\, F_{x,k}]^\top$$

The optimal control problem can then be described as

$$\begin{aligned}
\min_{\{x_k,u_k\}} \quad & t_N + \lambda_u \sum_{k=0}^{N-1} \|u_{k+1} - u_k\|_2^2 \\
\text{s.t.} \quad & x_{k+1} = x_k + \frac{\Delta s}{2}\big(f_k + f_{k+1}\big), \\
& u_{x,\min} \le u_{x,k} \le u_{x,\max}, \\
& \dot{s}_k \ge \varepsilon_s, \qquad 1 - \kappa_k e_k \ge \varepsilon_\kappa, \\
& -w_k + b_{\text{trk}} \le e_k \le w_k - b_{\text{trk}}, \\
& -\delta_{\max} \le \delta_k \le \delta_{\max}, \\
& F_{x,\min} \le F_{x,k} \le F_{x,\max}, \\
& d_{k,j}^2 \ge \rho_j^2, \qquad j \in \mathcal{J}_k, \\
& t_0 = 0, \qquad x_{0,i} = \bar{x}_{0,i}, \quad i \in \mathcal{I}_0.
\end{aligned}$$

Here $f_k = f_s(x_k, u_k; \kappa_k, \theta_k, \phi_k)$ denotes the spatial vehicle dynamics evaluated at node k, and

$$\dot{s}_k = \frac{u_{x,k}\cos d\psi_k - u_{y,k}\sin d\psi_k}{1 - \kappa_k e_k}.$$

The shorthand $d_{k,j}^2$ denotes the squared distance from the vehicle position $p(s_k, e_k)$ to obstacle j. The effective obstacle radius is

$$\rho_j = r_{\text{obs}}^{(j)} + r_{\text{veh}}$$

with the safety margin absorbed into the obstacle-clearance buffer used in the implementation.

The state components are body-frame longitudinal and lateral velocity (u_x, u_y), yaw rate r, longitudinal and lateral load-transfer states $(\Delta F_{z,\ell}, \Delta F_{z,a})$, elapsed time t, lateral track error e, and heading error $d\psi$ relative to the track centerline. The objective combines terminal time with a small control-difference regularizer. The constraints enforce collocation dynamics, forward progress, Frenet regularity, track limits, actuator bounds, and node-level obstacle clearance. In the implementation, obstacle clearance is also checked at intermediate segment samples. The Decision Transformer

does not replace this optimizer; instead, it predicts an initial state-control rollout that is either accepted and passed to the solver or rejected and replaced by the baseline initializer.

2.2. Decision Transformer formulation

The learned model is a Decision Transformer trained offline on expert trajectories produced by the optimizer. Following the sequence-modeling view of offline control (Chen et al., 2021), each trajectory is represented as a sequence of conditioning and prediction tokens derived from solver-generated rollouts. The DT is trained with a supervised autoregressive objective to predict the trajectory quantities needed to reconstruct a warm-start over the planning horizon.

The final model used in this project is a causal transformer with context length 50, 6 transformer layers, 4 attention heads, and embedding dimension 192. The model is trained on fixed offline datasets only; there is no online environment interaction or policy improvement loop. This is why the method is best understood as an offline RL approach implemented through sequence modeling rather than through value-function or policy-gradient machinery.

The return-to-go used for conditioning is defined from the spatial trajectory timing. If Δt_k is the time increment associated with segment k, the per-step reward is

$$r_k = -\Delta t_k,$$

and the return-to-go at step k is

$$R_k = \sum_{j=k}^{N-1} r_j = -\sum_{j=k}^{N-1} \Delta t_j.$$

Thus, larger return-to-go corresponds to less remaining traversal time. At training time, the model receives windows extracted from expert trajectories and is optimized to minimize next-token prediction error over the chosen trajectory representation. At deployment time, the model is rolled out autoregressively, so the relevant evaluation criterion is not teacher-forced prediction accuracy but solver-facing warm-start utility.

2.3. DT warm-starting for autonomous racing

At deployment time, the DT is used as a warm-start generator. Given the scenario context and the selected conditioning inputs, the model autoregressively produces a candidate trajectory over the planning horizon. This candidate is then passed through a wrapper that performs rollout validation and optional projection before deciding whether the warm-start should be submitted to the nonlinear solver.

The wrapper serves two purposes. First, it filters out clearly invalid trajectories whose rollout violates basic feasibility or produces large geometric inconsistencies. Second, it optionally applies simple corrective projections intended to reduce avoidable local errors before solver handoff. The resulting warm-start is accepted only if it satisfies a gate based on rollout validity and geometric burden. When the learned rollout is rejected, the system falls back to the baseline initialization used by the solver.

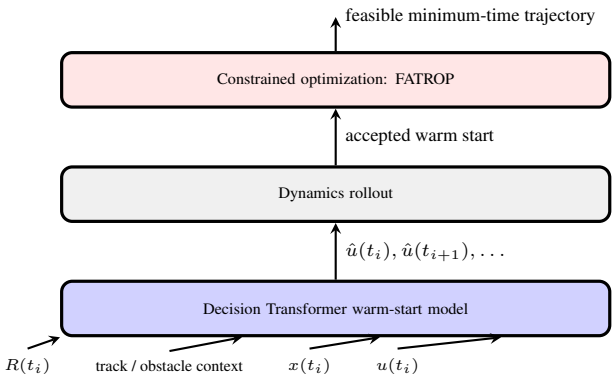

Figure 2. Warm-start architecture used in this project. Offline solver-generated trajectories train the DT. At deployment, the DT generates an autoregressive candidate trajectory that is validated, optionally projected, and either accepted as the solver warm-start or replaced by the baseline initializer.

This architecture preserves the role separation between learning and optimization. The DT supplies a structured initial guess; the solver remains responsible for satisfying hard constraints and refining the final trajectory. The overall warm-start pipeline is illustrated in Figure 2. The main empirical question is whether the learned initialization improves the operating point of the downstream optimizer relative to the baseline initializer.

2.4. Targeted repair-data fine-tuning

A central challenge in warm-starting is that autoregressive deployment exposes errors that are weakly represented in nominal expert data. To address this mismatch, the training pipeline augments the original optimizer-generated dataset with targeted repair data.

The first class of interventions consists of repair trajectories generated from states or segments where rollout quality degrades, so that the model is exposed to corrective behavior beyond the nominal distribution. A second class includes post-projection data, which is intended to align the model with the corrected trajectories produced by the wrapper. A final class uses failure-conditioned shards built from rollout failures observed under deployment, with the goal of concentrating supervision on the errors most responsible for rejection or poor solver handoff.

These interventions are not intended to change the downstream optimization problem. Instead, they test whether

Table 1. Representative training progression across the main DT variants. Validation loss is the best logged value for each checkpoint.

Model / Run	Data Regime	Val. Loss	Note
Larger-context parent	nominal + hard-repair	0.002835	Best parent checkpoint
Post-projection fine-tune	+ post-proj. repair	0.003170	Best deployment setting before failure FT
Lateral-stability fine-tune	+ lateral weighting	0.002773	Improved offline stability
Failure-conditioned fine-tune	+ failure-conditioned repair	0.002364	Best offline checkpoint

training on deployment-relevant corrections can reduce the gap between offline imitation quality and warm-start utility. The final comparison therefore focuses on whether such fine-tuning improves solver-facing metrics, not only validation loss.

3. Experimental Setup

3.1. Training data and model variants

The training set is derived from trajectories generated by the nonlinear trajectory optimizer on the target Oval-track scenarios. These trajectories provide expert demonstrations under the same vehicle model and constraint structure used downstream. The final dataset combines four shards: 31,710 nominal solver trajectories, 400 hard-repair trajectories, 1,000 post-projection repair trajectories, and 300 failure-conditioned repair trajectories. The repair shards were introduced to expose the model to rollout states that do not appear in the nominal expert distribution.

The main model variants follow a progression from nominal offline training to increasingly deployment-aware fine-tuning. Earlier runs train on nominal data alone. Later runs incorporate larger context windows and targeted repair shards, including post-projection and failure-conditioned examples. The purpose of this progression is not to compare many unrelated models, but to test a specific hypothesis: whether better offline trajectory modeling, especially on rollout-relevant failure modes, leads to improved downstream solver utility.

3.2. Training configuration

Training is performed offline on fixed datasets of solver-generated trajectories using the architecture described in Section 2.2. The training pipeline is implemented in PyTorch, optimized with AdamW, logged through TensorBoard/JSON artifacts, and configured for CUDA execution with mixed precision in the final GPU training scripts. Validation loss is tracked throughout training and later used only as an intermediate model-quality metric rather than as the primary success criterion. The main progression of training interventions is summarized in Table 1.

3.3. Deployment pipeline and metrics

Each model is evaluated as a warm-start generator for the downstream nonlinear solver. In the final project phase, the optimizer used for expert generation and warm-start evaluation is FATROP. For a given benchmark instance, the DT produces an autoregressive candidate trajectory, which is validated by the wrapper, optionally projected, and either accepted or rejected. Accepted trajectories are submitted to FATROP as warm-starts; rejected trajectories trigger fallback to the baseline initializer.

The primary deployment metrics are chosen to measure solver utility rather than imitation quality. These include:

- warm-start acceptance rate,
- solver success rate,
- solver iterations,
- solver wall-clock time,
- total pipeline time including DT inference.

Additional diagnostic metrics quantify projection burden and rollout validity, helping distinguish between trajectories that are merely accepted and those that materially improve the downstream solve.

Offline validation loss is also reported, but only as an intermediate metric. The central question is whether lower validation loss corresponds to lower downstream optimization cost.

3.4. Benchmarks and evaluation scope

The evaluation uses both obstacle-free and obstacle scenarios on the target racing track. The main controlled benchmark consists of corrected 10-scenario gates for the larger-context parent and post-projection fine-tune. A later smoke benchmark evaluates the failure-conditioned model on 3 obstacle-free and 3 obstacle scenarios after the final debugging pass. These experiments are intentionally narrow in scope: they focus on a fixed vehicle model, a single track family, and a limited set of benchmark scenarios chosen to test whether DT warm-starts provide a practical advantage over the baseline initializer in this setting. The main benchmark constants are summarized in Table 2.

4. Empirical Findings

The central empirical result of the project is that substantial offline validation improvements did not translate into a measurable downstream warm-start advantage.

This section reports three complementary views of deployment. First, the default-gate benchmark tests whether DT

Table 2. Main experiment parameters for the final Oval-track benchmark. Vehicle parameters come from the GTI model configuration used throughout the project.

Parameter	Value
Track	Oval_Track_260m
Track length	260 m
Track width	6.0 m
Planning horizon	$N = 120$ nodes
Vehicle	Volkswagen Golf GTI single-track model
Vehicle mass	1868 kg
Wheelbase	2.63 m
Max steering angle	27 deg
Longitudinal force bounds	$[-15,\ 10]$ kN
Road condition	Dry asphalt
Nominal tire friction	$\mu_f = 0.87,\ \mu_r = 1.03$
Obstacle scenarios	0 or 1–4 circular obstacles
Obstacle radius range	0.8–1.5 m
Obstacle margin	0.3 m
Obstacle clearance buffer	0.3 m
Solver	FATROP

warm-starts help in the intended pipeline, where only low-burden rollouts are accepted. Second, a relaxed-gate diagnostic tests whether the negative result is explained only by conservative rejection. Third, a small final smoke test checks whether the same conclusion persists for the best offline model after the main correctness fixes. Throughout, we distinguish *solver-only* runtime from *total* runtime: a DT rollout may slightly reduce the nonlinear solver's raw solve time while still increasing end-to-end runtime once warm-start inference is included.

4.1. Offline modeling improved substantially

Across later training runs, offline validation performance improved materially relative to the earlier baseline models. Larger-context training reduced sequence-model error, and subsequent targeted fine-tuning on repair and failure-conditioned data further improved validation loss. These trends suggest that the DT successfully learned more of the structure present in the optimizer-generated trajectories, especially when training data was augmented to reflect rollout-relevant corrections.

From an offline modeling perspective, the failure-conditioned runs were therefore the strongest models in the study. If offline sequence quality were a reliable proxy for downstream warm-start utility, these later models would be the leading candidates for deployment benefit. Figure 3 shows a representative training curve for one of the later fine-tuning runs and supports the narrower claim that training remained stable under the later configuration.

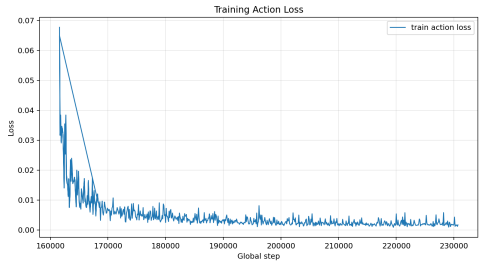

Figure 3. Training action-loss curve for the post-projection fine-tune. The figure is included only as evidence that the later fine-tuning configuration trained stably.

4.2. Improved offline performance did not yield warm-start utility

Despite these offline gains, the deployment results do not show a convincing improvement over the baseline solver initialization. Under the default acceptance gate, warm-start acceptance remained near zero for the main DT variants, which meant that the downstream solver usually fell back to the baseline initializer. In this regime, solver iterations remained unchanged relative to baseline, and although raw solver time was sometimes slightly lower, total runtime was typically worse once DT inference time was included.

This mismatch between offline improvement and deployment utility is the main empirical lesson of the paper. The learned models became better predictors of optimizer-generated trajectories under offline evaluation, but this did not place the downstream solver in a meaningfully better operating region during deployment. Representative corrected deployment numbers are reported in Table 3.

4.3. Relaxed acceptance did not recover solver benefit

A natural alternative explanation is that the default wrapper gate was simply too strict. To test this, a relaxed-gate diagnostic was used to admit some trajectories that would otherwise have been rejected. This increased warm-start acceptance, showing that the default gate was not the only barrier to solver handoff.

However, the accepted relaxed-gate trajectories still did not reduce solver work on average. On the corrected 10-scenario gates, acceptance increased to 20%, but mean solver iterations increased from 191.8 to 206.8 on the no-obstacle gate and from 230.7 to 256.0 on the obstacle gate. Total runtime also remained above baseline once inference cost was included. This result is important because it shows that warm-start acceptance alone is not enough. The learned trajectory must not only pass the wrapper, but must place the solver close enough to a useful basin of attraction to reduce the downstream optimization burden.

Table 3. Representative downstream results on the Oval track. The first two rows use the corrected 10-scenario default-gate obstacle benchmark. The next two rows use corrected 10-scenario relaxed-gate diagnostics, shown only to test whether low acceptance alone explains the negative result. The final row is a 3-scenario default-gate smoke benchmark for the failure-conditioned model after the major correctness fixes. Negative Solve Δ means lower *solver-only* runtime than baseline; positive Total Δ means slower *end-to-end* runtime once DT inference is included.

Model / Evaluation Setting	Val. Loss	Accept.	Iter. Δ	Solve Δ	Total Δ
Parent DT (default gate, obstacle benchmark)	0.002835	0%	0.0	−2.34s	+0.36s
Post-proj. FT (default gate, obstacle benchmark)	0.003170	0%	0.0	−1.90s	+0.73s
Post-proj. FT (relaxed gate, no-obstacle diagnostic)	0.003170	20%	+15.0	+0.06s	+2.92s
Post-proj. FT (relaxed gate, obstacle diagnostic)	0.003170	20%	+25.3	+1.55s	+4.43s
Failure-conditioned FT (default gate, obstacle smoke test)	0.002364	0%	0.0	−2.38s	+0.33s

4.4. Qualitative rollout behavior

A useful complement to the quantitative tables is a qualitative comparison between the raw DT rollout and the solver-refined trajectory on an obstacle scenario. Figure 4 shows a representative obstacle case from the final failure-conditioned evaluation. The DT rollout captures the broad maneuver, but it remains offset from the solver-refined obstacle-avoidance line, illustrating why visually plausible rollouts can still fail to provide a useful warm-start.

4.5. Summary of empirical findings

Taken together, the experiments support three conclusions. First, larger-context and targeted fine-tuning improve offline sequence modeling. Second, those gains do not translate into a convincing solver-facing advantage under the corrected deployment benchmarks. Third, the gap is not explained solely by a conservative acceptance gate, since relaxed acceptance admits some trajectories without recovering downstream benefit.

5. Failure Analysis

5.1. Corrected evaluations did not change the final conclusion

Several implementation and evaluation issues were identified and corrected during the project, including benchmark confounds and rollout-validation inconsistencies. These fixes mattered because some intermediate readings were either too optimistic or too pessimistic. After the corrected evaluation path was established, however, the overall conclusion did not change: no DT variant showed a clear downstream warm-start advantage over the baseline initializer.

This point matters because it distinguishes the final result from a simple implementation failure. The project did not stop at an inconclusive benchmark; the evaluation pipeline was revised and the main deployment tests were repeated. The conclusions reported here are therefore based on the corrected system rather than on preliminary intermediate results.

5.2. Early lateral rollout behavior explains the remaining gap

The strongest remaining explanation for the deployment gap is that the DT still makes poor early lateral commitments under autoregressive rollout. In practice, this appears as trajectories that move to the wrong side of the track or obstacle geometry too early, or with a lateral magnitude that is too aggressive to be solver-useful. These errors are often exposed downstream as repeated projection burden, clipping, or wrapper rejection.

This diagnosis is more specific than generic instability. The later models do not primarily fail because the rollout is globally incoherent everywhere; instead, they often fail because the trajectory enters the wrong local region early in the horizon. Once that happens, later corrections are not sufficient to produce a warm-start that helps the solver. Even accepted trajectories can remain solver-harmful if their early geometry is inconsistent with the basin the optimizer would otherwise exploit.

Figure 4 provides a representative qualitative example. The DT rollout captures the broad maneuver but commits early to the wrong lateral branch or moves outward too aggressively, after which clipping and projection expose the mismatch.

5.3. Implications for evaluating learned warm-starts

This failure mode explains why offline validation loss was a weak predictor of deployment success. Average sequence-model improvement does not guarantee that the rollout makes the right early geometric decisions under autoregressive deployment. In warm-starting applications, these local early-horizon choices can matter more than aggregate token-level accuracy.

More broadly, the results suggest that evaluating learned warm-starts requires deployment metrics that directly mea-

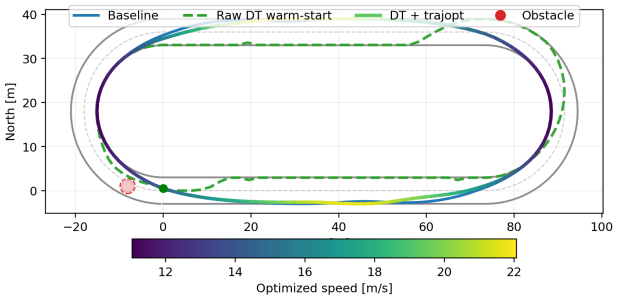

Figure 4. Representative obstacle-avoidance comparison between the DT rollout and the solver-refined trajectory from the final failure-conditioned evaluation.

sure solver utility. Validation loss, acceptance rate, and even qualitative rollout plausibility can all improve without yielding reduced downstream optimization cost. In this project, the main unresolved challenge is therefore not generic forecasting quality, but early lateral basin selection under rollout.

6. Limitations

The conclusions of this project should be interpreted in light of several limitations. First, the empirical evaluation is deliberately narrow in scope. The experiments focus on a single racing setting with a fixed vehicle model, limited benchmark families, and relatively small corrected deployment suites. A larger and more diverse benchmark set would strengthen the generality of the conclusions.

Second, the deployment pipeline depends on a hand-designed wrapper that performs validation, optional projection, and acceptance gating before solver handoff. Although relaxed-gate diagnostics help separate gate conservatism from rollout quality, the precise choice of wrapper logic still influences the observed acceptance and failure patterns.

Third, the final diagnosis is based on observed rollout behavior and solver-facing metrics rather than on a formal theoretical characterization of optimizer basins or autoregressive error propagation. The claim that early lateral rollout behavior is the dominant bottleneck is therefore an evidence-based empirical interpretation rather than a formal proof.

These limitations weaken the strength of any broad positive or negative generalization, but they do not overturn the central result of the paper: in the corrected experiments reported here, improved offline sequence-model quality did not yield a convincing downstream warm-start benefit.

7. Conclusion

This project evaluated whether a Decision Transformer can provide useful warm-start trajectories for nonlinear minimum-time vehicle trajectory optimization with obstacle avoidance. The resulting system combined offline sequence modeling of optimizer-generated trajectories with autoregressive rollout, wrapper-based validation and projection, and downstream nonlinear solver refinement.

Empirically, larger-context training and targeted repair-data fine-tuning substantially improved offline validation performance. However, these improvements did not produce a convincing solver-facing advantage in the corrected deployment benchmarks. Under the default gate, acceptance remained near zero; under relaxed acceptance, some trajectories were admitted, but they still did not reduce solver work on average. The final evidence therefore indicates that offline sequence-model quality was not a reliable proxy for warm-start utility in this setting.

The most plausible remaining bottleneck is early lateral rollout behavior. The learned model often failed not because of global incoherence, but because it entered the wrong local region of the trajectory space too early for the downstream solver to benefit. Future work should therefore focus less on improving average offline prediction quality and more on directly targeting deployment-critical early-horizon decisions and solver-relevant rollout geometry.

8. Author Contributions and Code

Author contributions Victor Sebastian Martinez Perez: sole project member.

Code https://github.com/vSebas/CS234_Final_Project

References

Andersson, J. A. E., Gillis, J., Horn, G., Rawlings, J. B., and Diehl, M. CasADi: A software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, 11(1):1–36, 2019.

Celestini, D., Gammelli, D., Guffanti, T., D'Amico, S., Capello, E., and Pavone, M. Transformer-based model predictive control: Trajectory optimization via sequence modeling. *IEEE Robotics and Automation Letters*, 9(11): 9820–9827, 2024.

Chen, L., Lu, K., Rajeswaran, A., Lee, K., Grover, A., Laskin, M., Abbeel, P., Srinivas, A., and Mordatch, I. Decision transformer: Reinforcement learning via sequence modeling. In *Advances in Neural Information Processing Systems*, volume 34, pp. 15084–15097, 2021.

Guffanti, T., Gammelli, D., D'Amico, S., and Pavone, M. Transformers for trajectory optimization with application to spacecraft rendezvous. In *IEEE Aerospace Conference*, pp. 1–15. IEEE, 2024.

Kapania, N. R., Subosits, J., and Gerdes, J. C. A sequential two-step algorithm for fast generation of vehicle racing trajectories. In *Proceedings of the ASME Dynamic Systems and Control Conference*, 2015. DSCC2015-9757.

Subosits, J. K. and Gerdes, J. C. From the racetrack to the road: Real-time trajectory replanning for autonomous driving. *IEEE Transactions on Intelligent Vehicles*, 2020.

Subosits, J. K. and Gerdes, J. C. Impacts of model fidelity on trajectory optimization for autonomous vehicles in extreme maneuvers. *IEEE Transactions on Intelligent Vehicles*, 6(3):546–557, 2021. doi: 10.1109/TIV.2021.3051325.

Waechter, A. and Biegler, L. T. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*, 106(1):25–57, 2006.